- POP Maker - 프로젝트 개요
  - 1. 프로젝트 소개
  - 2. 기술 스택
  - 3. 시스템 아키텍처
    - 핵심 데이터 흐름
  - 4. 주요 기능
    - 4.1 AI POP 생성 파이프라인
    - 4.2 인라인 에디터
    - 4.3 이미지 관리
    - 4.4 배경 관리
    - 4.5 내보내기
  - 5. 페이지 타입 & 테마
    - 5.1 페이지 타입 (8종)
    - 5.2 시즌 테마 (5종)
    - 5.3 섹션 타입 (17종)
  - 6. 프로젝트 구조
  - 7. DB 스키마
    - projects
    - section_templates
    - project_images
    - users
  - 8. API 엔드포인트 요약
    - 인증 (/api/auth)
    - 프로젝트 (/api/projects)
    - 생성 (/api/generate)
    - 이미지 (/api/images)
    - 업로드 (/api/upload)
    - 참조 데이터
  - 9. 개발 환경 설정
    - 사전 요구사항
    - Docker Compose로 실행 (권장)
    - 수동 실행
    - 환경변수
  - 10. 배포
    - 인프라
    - 배포 플로우
    - 수동 배포

POP Maker - 프로젝트 개요

1. 프로젝트 소개

POP Maker는 신세계백화점 대상 POP(Point of Purchase) 마케팅 콘텐츠를 AI로 자동 생성하는 웹 애플리케이션입니다.

상품 정보와 이미지를 입력하면 AI가 카피라이팅, 이미지 편집, 레이아웃 구성을 자동으로 수행하여 즉시 사용 가능한 웹 기획전 페이지를 생성합니다.

핵심 가치:

- 기획전 제작 시간 대폭 단축 (수일 → 수분)
- 일관된 디자인 품질 유지
- 비디자이너도 전문적인 콘텐츠 제작 가능

---

2. 기술 스택

레이어	기술	버전
Frontend	Next.js (App Router)	14.2.35
	React + TypeScript	18 / 5
	Tailwind CSS	3.4.1
	Zustand (상태관리)	5.0.10
Backend	FastAPI + Pydantic	0.115.6 / 2.10.5
	Python	3.11+
	Uvicorn (ASGI)	0.34.0
Database	Supabase (PostgreSQL)	-
	Supabase Storage	-
AI - 텍스트/이미지	Google Gemini 2.5 Flash	google-genai 1.0+
AI - 누끼 제거	AWS Bedrock Nova Canvas	amazon.nova-canvas-v1:0

레이어	기술	버전
인증	JWT (PyJWT)	2.10.1
배포	AWS EC2 + GitHub Actions	-
컨테이너	Docker Compose	-

3. 시스템 아키텍처

```
┌──────────────────────────────────────────────────────────┐
│                     Frontend (Next.js)                    │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│  │ 입력 폼  │→│ 생성 요청 │→│  에디터  │→│ 내보내기│  │
│  └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
└──────────────────────────────────────────────────────────┘
                          │ REST API
                          ▼
┌──────────────────────────────────────────────────────────┐
│                     Backend (FastAPI)                     │
│  ┌──────────────────────────────────────────────────┐    │
│  │              Generate Orchestrator                │    │
│  │  [섹션 결정] → [AI 텍스트] → [AI 이미지] → [렌더링]│    │
│  └──────────────────────────────────────────────────┘    │
│        │              │              │                    │
│  ┌─────▼─────┐  ┌─────▼─────┐  ┌─────▼─────┐             │
│  │ Template  │  │ Template  │  │ BgRemove  │             │
│  │ Render Svc│  │ AI Svc    │  │ Service   │             │
│  └───────────┘  └─────┬─────┘  └─────┬─────┘             │
└──────────────────────┼──────────────┼──────────────────┘
                       │              │
                  ┌────▼─────────┐  ┌─▼──────────────┐
                  │ Google Gemini│  │ AWS Bedrock    │
                  │ 2.5 Flash    │  │ Nova Canvas    │
                  └──────────────┘  └────────────────┘

                  ┌──────────────────────────┐
                  │  Supabase (PostgreSQL)   │
                  │  + Supabase Storage      │
                  └──────────────────────────┘
```

핵심 데이터 흐름

```
사용자 입력 (상품정보, 이미지, 페이지타입, 테마)
    ↓
POST /api/generate
    ↓
```

```
[1] 페이지 타입 기반 섹션 자동 결정
[2] Gemini로 섹션별 카피 생성
[3] Gemini로 섹션별 이미지 생성/편집
[4] 배경 처리 (단색 or AI 생성)
[5] HTML 템플릿 렌더링 + 데이터 바인딩
    ↓
rendered_sections[] → DB 저장
    ↓
프론트엔드 에디터에서 인라인 편집 → PNG/JPG 내보내기
```

---

4. 주요 기능

4.1 AI POP 생성 파이프라인

5단계 자동 생성 파이프라인:

단계	설명	기술
1. 섹션 결정	페이지 타입 + 상품 수 기반 섹션 조합 자동 결정	룰 기반
2. AI 텍스트	섹션별 카피라이팅 (타이틀, 설명, 배지 등)	Gemini 2.5 Flash
3. AI 이미지	섹션 컨텍스트 기반 이미지 생성/편집	Gemini Image Editor
4. 배경 처리	단색 배경 또는 AI 배경 이미지 생성	Gemini / Solid Color
5. 렌더링	HTML 템플릿에 데이터 바인딩 + 스타일 적용	Template Render Service

4.2 인라인 에디터

- 텍스트 직접 편집 (클릭하여 수정)
- 플로팅 텍스트 툴바 (폰트, 크기, 색상, 정렬)
- 섹션 순서 드래그 앤 드롭
- 섹션별 속성 패널
- Undo/Redo (무제한)

4.3 이미지 관리

- 상품 이미지 드래그 앤 드롭 업로드
- **누끼 제거** (AWS Bedrock Nova Canvas)

    - 자동 배경 제거 → 투명 PNG
    - 피사체 중앙 정렬, 가장자리 잘림 방지 패딩
    - 이미지 크기 자동 조정 (320~4096px)
- 채팅 기반 AI 이미지 재생성 (커스텀 프롬프트)

4.4 배경 관리

- **적용 범위**: 글로벌 (전체 동일) / 섹션별 개별 설정
- **배경 타입**: 단색 (HEX) / AI 생성 / 없음
- 배경 투명도, 밝기 조절
- 배경색 기반 동적 텍스트 색상 (W3C 휘도 공식)

4.5 내보내기

- PNG / JPG 이미지 내보내기 (html-to-image)
- 섹션 단위 또는 전체 페이지 내보내기

---

5. 페이지 타입 & 테마

5.1 페이지 타입 (8종)

ID	이름	상품 수	가격 필수	생성 섹션 구성
`product_detail`	상품 포인트	2-6	-	hero_banner + feature_badges + description + feature_point ×N
`promotion`	상품 기획전	2-6	O	promo_hero + product_card ×N
`brand_promotion`	브랜드 기획전	3 (고정)	O	fit_hero + fit_event_info + fit_brand_special + fit_product_trio
`vip_special`	VIP 스페셜위크	2-6	O	vip_special_hero + product_card ×N

ID	이름	상품 수	가격 필수	생성 섹션 구성
vip_private	VIP 프라이빗위크	2-6	O	vip_private_hero + product_card ×N
gourmet	맛집 기획전	1-5 (식당)	-	gourmet_hero + gourmet_restaurant ×N + (와인 섹션)
shinsegae	뱅드신세계	2-6	O	shinsegae_hero + product_card ×N
custom	섹션 직접 선택	1-9	-	사용자 선택 섹션 조합

5.2 시즌 테마 (5종)

ID	이름	아이콘	액센트 컬러
holiday	홀리데이	🎄	#C41E3A
spring_sale	봄 세일	🌸	#E91E90
summer_special	여름 특가	☀️	#0099FF
autumn_new	가을 신상	🍂	#D2691E
winter_promo	겨울 프로모션	❄️	#4169E1

각 테마는 고유한 배경 AI 프롬프트와 카피 키워드를 포함합니다.

5.3 섹션 타입 (17종)

```
hero_banner          — 히어로 배너
feature_badges       — 특징 배지
description          — 텍스트 설명
feature_point        — 상품 포인트
promo_hero           — 프로모션 히어로
product_card         — 상품 카드
fit_hero             — 브랜드 기획전 히어로
fit_event_info       — 이벤트 정보
fit_brand_special    — 브랜드 스페셜
fit_product_trio     — 3상품 레이아웃
vip_special_hero     — VIP 스페셜 히어로
vip_private_hero     — VIP 프라이빗 히어로
```

```
gourmet_hero         - 맛집 히어로
gourmet_restaurant   - 레스토랑 정보
gourmet_wine_intro   - 와인 소개
gourmet_wine         - 와인 상세
shinsegae_hero       - 뱅드신세계 히어로
```

---

6. 프로젝트 구조

```
POP_maker/
├── frontend/                          # Next.js 14 프론트엔드
│   ├── app/                           # App Router 페이지
│   │   ├── page.tsx                   # 홈 (프로젝트 목록)
│   │   ├── login/page.tsx             # 로그인
│   │   └── result/[projectId]/        # 에디터 페이지
│   ├── components/
│   │   ├── editor/                    # 에디터 컴포넌트 (11개)
│   │   │   ├── SectionRenderer.tsx    # 섹션 렌더링 + 줌
│   │   │   ├── SectionBlock.tsx       # 개별 섹션 컨테이너
│   │   │   ├── SectionListSidebar     # 섹션 목록 사이드바
│   │   │   ├── TextToolbar.tsx        # 플로팅 텍스트 툴바
│   │   │   ├── ChatPanel.tsx          # AI 이미지 재생성 채팅
│   │   │   ├── BackgroundPanel.tsx    # 배경 설정 패널
│   │   │   ├── PropertyPanel.tsx      # 속성 편집 패널
│   │   │   └── ImagePanel.tsx         # 이미지 관리 패널
│   │   ├── forms/                     # 입력 폼 (8개)
│   │   ├── auth/                      # 인증 컴포넌트
│   │   ├── layout/                    # 레이아웃
│   │   ├── modals/                    # 모달 다이얼로그
│   │   ├── preview/                   # 미리보기
│   │   ├── export/                    # 내보내기
│   │   └── ui/                        # UI 프리미티브
│   ├── stores/
│   │   └── editorStore.ts             # Zustand 에디터 상태
│   ├── hooks/
│   │   └── useHistory.ts              # Undo/Redo 훅
│   ├── lib/
│   │   ├── api.ts                     # API 클라이언트
│   │   ├── supabase.ts                # Supabase 설정
│   │   ├── export.ts                  # 이미지 내보내기
│   │   └── fonts.ts                   # 폰트 유틸리티
│   └── types/index.ts                 # 전체 타입 정의
│
├── backend/                           # FastAPI 백엔드
│   ├── app/
│   │   ├── main.py                    # FastAPI 앱 진입점 (v5.3.0)
│   │   ├── config.py                  # 환경 설정 (Pydantic Settings)
│   │   ├── database.py                # Supabase 클라이언트
│   │   ├── routers/                   # API 라우터 (8개)
│   │   │   ├── auth.py                # 인증/인가
│   │   │   ├── projects.py            # 프로젝트 CRUD + 섹션 수정
```

```
│   │   │   ├── generate.py          # POP 생성 파이프라인
│   │   │   ├── upload.py            # 이미지 업로드
│   │   │   ├── images.py            # 이미지 관리 + 누끼 제거
│   │   │   ├── templates.py         # 섹션 템플릿 조회
│   │   │   ├── themes.py            # 테마 조회
│   │   │   └── page_types.py        # 페이지 타입 조회
│   │   ├── services/                # 비즈니스 로직 (6개)
│   │   │   ├── generate_orchestrator.py  # 생성 파이프라인 오케스트레이터
│   │   │   ├── template_ai_service.py    # Gemini AI 텍스트/이미지 생성
│   │   │   ├── template_render_service.py # HTML 템플릿 렌더링
│   │   │   ├── bg_remove_service.py      # Bedrock 누끼 제거
│   │   │   ├── storage_service.py        # Supabase Storage 관리
│   │   │   └── section_compose_service.py # 섹션 구성
│   │   ├── schemas/                 # Pydantic 스키마
│   │   ├── constants/               # 상수 (페이지타입, 테마, 프롬프트)
│   │   ├── prompts/                 # AI 프롬프트 템플릿
│   │   └── dependencies/auth.py     # JWT 인증 미들웨어
│   └── requirements.txt
│
├── supabase/migrations/             # DB 마이그레이션 (30+ SQL)
├── scripts/run_migrations.py        # 마이그레이션 실행 스크립트
├── .github/workflows/
│   └── deploy-backend.yml           # EC2 자동 배포
├── docker-compose.yml               # 로컬 개발 환경
└── .env.example                     # 환경변수 템플릿
```

---

7. DB 스키마

projects

컬럼	타입	설명
`id`	UUID PK	프로젝트 ID
`user_id`	UUID FK	사용자 ID
`brand_name`	varchar	브랜드명
`theme_id`	varchar	테마 ID
`template_used`	varchar	사용 템플릿
`products`	jsonb	상품 배열 [{name, price, description, image_url}]
`restaurants`	jsonb	레스토랑 배열 (고메 전용)
`selected_sections`	text[]	선택된 섹션 타입 배열

컬럼	타입	설명
rendered_sections	jsonb	렌더링된 섹션 배열 (HTML+CSS+data)
generated_data	jsonb	AI 생성 텍스트/프롬프트
input_data	jsonb	사용자 입력 원본
pipeline_result	jsonb	파이프라인 결과 메타데이터
background_config	jsonb	배경 설정
background_settings	jsonb	배경 추가 설정 (scope, opacity 등)
created_at	timestamp	생성일시
updated_at	timestamp	수정일시

section_templates

컬럼	타입	설명
id	UUID PK	템플릿 ID
section_type	varchar	섹션 타입
name	varchar	템플릿 이름
html_template	text	HTML 템플릿 (placeholder 포함)
css_template	text	CSS 스타일
placeholders	jsonb	placeholder 정의 배열
is_active	boolean	활성화 여부

project_images

컬럼	타입	설명
id	UUID PK	이미지 ID
project_id	UUID FK	프로젝트 ID
image_type	varchar	타입 (input / generated / bg_removed)
storage_path	varchar	Storage 경로

컬럼	타입	설명
`original_filename`	varchar	원본 파일명
`sort_order`	integer	정렬 순서
`created_at`	timestamp	생성일시

users

컬럼	타입	설명
`id`	UUID PK	사용자 ID
`username`	varchar (unique)	로그인 ID
`password_hash`	varchar	SHA256 해시
`display_name`	varchar	표시 이름
`is_admin`	boolean	관리자 여부
`created_at`	timestamp	생성일시

---

8. API 엔드포인트 요약

인증 (`/api/auth`)

Method	Path	설명	인증
POST	`/login`	로그인 (JWT 발급)	-
POST	`/verify`	토큰 검증	O
POST	`/logout`	로그아웃	O
POST	`/users`	계정 생성	Admin
GET	`/users`	계정 목록	Admin
DELETE	`/users/{user_id}`	계정 삭제	Admin

프로젝트 (`/api/projects`)

Method	Path	설명	인증
POST	`/`	프로젝트 생성	O
GET	`/`	프로젝트 목록	O
GET	`/{id}`	프로젝트 상세	O
PUT	`/{id}`	프로젝트 수정	O
DELETE	`/{id}`	프로젝트 삭제	O
PUT	`/{id}/sections/{section_id}/data`	섹션 데이터 수정	O
POST	`/{id}/sections/{section_id}/regenerate-image`	이미지 재생성	O
POST	`/{id}/generate-background`	배경 AI 생성	O

생성 (`/api/generate`)

Method	Path	설명	인증
POST	`/`	POP 생성 파이프라인 실행	O

이미지 (`/api/images`)

Method	Path	설명	인증
GET	`/project/{project_id}`	프로젝트 이미지 목록	O
POST	`/remove-bg`	배경 제거 (누끼)	O

업로드 (`/api/upload`)

Method	Path	설명	인증
POST	`/`	이미지 업로드	O

참조 데이터

Method	Path	설명	인증
GET	`/api/templates`	섹션 템플릿 목록	-
GET	`/api/templates/{section_type}`	특정 섹션 템플릿	-
GET	`/api/themes`	테마 목록	-
GET	`/api/page-types`	페이지 타입 목록	-
GET	`/health`	헬스체크	-

---

9. 개발 환경 설정

사전 요구사항

- Node.js 20+
- Python 3.11+
- Docker & Docker Compose (선택)

Docker Compose로 실행 (권장)

```
# 환경변수 설정
cp .env.example .env
# .env 파일에 실제 키 입력

# 실행
docker-compose up
```

- Frontend: http://localhost:3000
- Backend: http://localhost:8000

수동 실행

```
# Backend
cd backend
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend
cd frontend
npm install
npm run dev
```

환경변수

```
# 필수
SUPABASE_URL=              # Supabase 프로젝트 URL
SUPABASE_SERVICE_KEY=      # Supabase 서비스 키
GEMINI_API_KEY=            # Google Gemini API 키
AWS_BEARER_TOKEN_BEDROCK= # AWS Bedrock 토큰
AUTH_SECRET=               # JWT 서명 키

# 프론트엔드
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
```

---

10. 배포

인프라

- **서버**: AWS EC2 (ap-southeast-2)
- **CI/CD**: GitHub Actions (`.github/workflows/deploy-backend.yml`)
- **터널**: Cloudflare Quick Tunnel (프론트엔드 노출)

배포 플로우

```
main 브랜치 push (backend/** 변경)
    ↓ GitHub Actions
서버 프로세스 중지 → 파일 전송 (SCP) → 의존성 설치 → 서버 시작 → 헬스체크
```

수동 배포

```
ssh -i ssg-pdp-poc.pem ec2-user@3.26.91.92
cd /home/ec2-user/backend
sudo pkill -9 -f "python3.11.*uvicorn"
sudo nohup python3.11 -m uvicorn app.main:app \
  --host 0.0.0.0 --port 8000 > nohup.out 2>&1 &
```